

FLOWMESH: A SERVICE FABRIC FOR COMPOSABLE LLM WORKFLOWS

Junyi Shen¹ Noppanat Wadlom¹ Lingfeng Zhou² Dequan Wang² Xu Miao³ Lei Fang³ Yao Lu¹

ABSTRACT

AI deployment increasingly resembles a pipeline of data transformation, fine-tuning, and agent interactions rather than a monolithic LLM job; recent examples include RLHF/RLAIF training and agentic workflows. To cope with this shift, we propose FlowMesh, a multi-tenant service fabric that executes and optimizes these workloads as one shared service instead of isolated pipelines. It decomposes workflows into fine-grained operators with recorded lineage, enabling de-duplication of work across users and batching requests on the same hardware while preserving per-workflow provenance. A global control plane maintains a cluster-wide pool of ready operators and uses a single utility function to pick both the batch and the worker, balancing throughput, cost, and data locality on heterogeneous GPUs. The data plane is an elastic fleet of stateless workers backed by a content-addressable store, enabling rapid, automatic scale-out, safe retry after preemption, and portability across managed clusters such as Kubernetes and geo-distributed GPU marketplaces such as Vast.ai. Compared with baseline solutions, FlowMesh achieves up to 3.8× cost reduction and 2.0× lower energy usage, provides a similar or better latency profile, and remains efficient under dynamic and failure-prone conditions.

1 INTRODUCTION

The dominant paradigm in applied machine learning is undergoing a significant transformation, shifting from monolithic, single-shot model training and inference to the orchestration of complex, multi-stage post-training workflows. These workflows, encompassing methodologies like Reinforcement Learning from Human Feedback (RLHF) (Ouyang et al., 2022), LLM-as-a-Judge (Gu et al., 2024), Reinforcement Learning from AI Feedback (RLAIF) (Lee et al., 2024), multi-agent systems and agentic workflows (Yao et al., 2023), are now the primary drivers of LLM research and production. Such processes are naturally expressed as Directed Acyclic Graphs (DAGs) of fine-grained, interdependent tasks, including data preparation, supervised fine-tuning, reward modeling, policy fine-tuning, and evaluation. This inherent graph structure facilitates decades of prior work on compilers and query optimizers for substantial system-level optimizations.

However, a critical mismatch exists between the graph-structured, multi-tenant nature of modern AI development and the execution models of contemporary cluster management and processing solutions. Platforms like Kubernetes and Spark, while proficient at managing containerized workloads, treat LLM jobs as opaque, black-box entities (Verma

et al., 2015; Zaharia et al., 2012). They lack the semantic understanding of the internal DAG structure, precluding any form of cross-job optimization. This fundamental disconnect gives rise to three critical challenges that severely hinder efficiency, scalability, and cost-effectiveness in a GPU cluster or network:

Redundant Compute: In a typical multi-tenant system, multiple users submit jobs with similar workflows for concurrent execution, such as fine-tuning the base model on slightly varied datasets, or serving the same base model with different LoRA adapters (Hu et al., 2022). Without a global awareness of the workload, existing systems compel each job to re-execute identical sub-tasks, like reward model inference or initial supervised fine-tuning (SFT), leading to a tremendous waste of expensive GPU cycles.

Heterogeneous Infrastructures: Production GPU clusters are inherently heterogeneous, comprising a mix of architectures (e.g., H100, A100) and varying interconnects. Naive scheduling fails to solve the complex, multi-objective optimization problem of matching diverse task requirements (e.g., CPU-bound data preprocessing, latency-sensitive inference, memory-bound training) to the most appropriate hardware. This mismatch also results in poor resource utilization, performance bottlenecks, and high operational costs.

Non-trivial Provisioning: The iterative, interactive nature of AI research and development generates highly bursty and unpredictable workloads. Statistically allocated resources, the norm in systems like Ray or Slurm-based setups (Moritz et al., 2018), present a stark trade-off: overprovisioning

¹National University of Singapore ²Shanghai Jiao Tong University ³DataCanvas. Correspondence to: Yao Lu <yao@nus.edu.sg>.

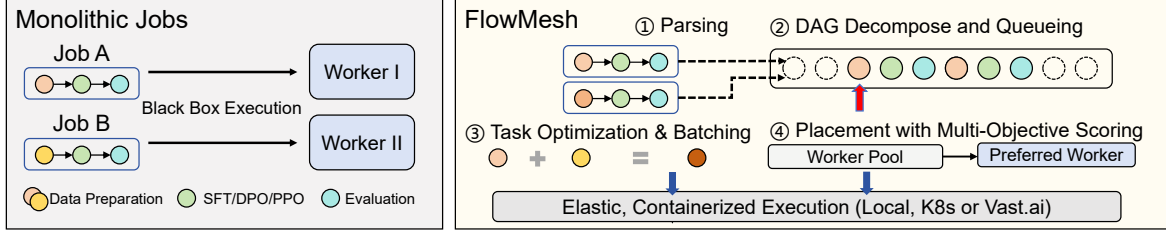


Figure 1. FlowMesh decomposes multi-stage LLM workflows into fine-grained, tasks and dispatch them among distributed workers.

leads to costly idle capacity, while underprovisioning results in prohibitive queueing delays that stifle productivity.

In a nutshell, the core issue is one of abstraction. Forcing a graph-structured workflow into a large, block-resource abstraction is inherently inefficient. It either necessitates over-provisioning for the peak resource needs of the entire workflow or introduces significant external orchestration complexity. An ideal system should consider inter-workflow dependencies and resource needs at a fine-grained, operator level, enabling optimizations that are impossible when the entire workflow is implemented in an opaque container.

In this paper, we propose FlowMesh, a service fabric that re-frames LLM post-training workflows from isolated jobs into composable, multi-tenant services upon distributed GPU workers. As demonstrated in Figure 1, FlowMesh applies the following key ideas: (1) *Workflow abstraction with containerized execution*: It encodes incoming workflows into DAGs, and uses a high-performance, containerized worker runtime to offset heterogeneity. (2) *Resource-aware placement and batching*: It uses a scheduler to make global placement and batching decisions that are topology- and resource-aware. (3) *Serverless compute*: It dynamically and automatically adjusts the worker pool to match demand and offer fault tolerance.

We implemented FlowMesh on a centralized Kubernetes GPU cluster as well as on a decentralized Vast.ai network (AI, 2025). Both backends run the same containerized workers pulled from a public registry and read/write data and logs to a shared content-addressable store. On Kubernetes, Pods are autoscaled and placed with GPU constraints; on Vast.ai, the control loop leases heterogeneous instances on demand and launches the identical image. Across both environments, the control plane schedules a queue of per-user DAGs, unifies identical operators, consolidates compatible ones into data locality-aware macro-batches, and delivers portable performance and cost efficiency without changing user workflows. On realistic multi-tenant workloads, FlowMesh matches or exceeds baseline throughput while reducing costs by up to 3.8 $\times$ and energy usage by up to 2.0 $\times$ at similar or better latency, and it stays efficient under dynamic load and failures.

To conclude, contributions of this paper include:

- We propose FlowMesh, a multi-tenant service fabric that runs LLM post-training and agentic workflows as a shared service instead of isolated pipelines.
- FlowMesh breaks workflows into fine-grained operators with lineage and uses a global control plane plus an elastic pool of stateless workers to batch compatible work, schedule across heterogeneous GPUs, and scale across Kubernetes and decentralized GPU markets.
- We evaluate FlowMesh on realistic multi-tenant workloads, and demonstrate significant performance improvements in throughput and cost efficiency.

2 BACKGROUND AND MOTIVATION

The motivation for FlowMesh stems from the massive economic cost of the status quo: high-end GPUs are a scarce and expensive resource, and a significant fraction of large cluster investments is wasted due to redundant computation and poor utilization. This inefficiency directly translates to wasted capital expenditure and reduced productivity. To ground our design, we first analyze the structure of modern ML workflows and then detail the shortcomings of existing systems to support these workflows.

The Anatomy of LLM Post-Training Workflows. To illustrate the inherent, composable graph structure and compute reuse opportunities, we deconstruct two canonical examples.

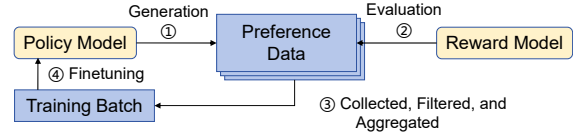


Figure 2. RLHF/RLAIF through reward and feedback.

RLHF Training: A typical RLHF pipeline (shown in Figure 2) is an iterative DAG consisting of several distinct stages. It begins with (1) parallel generation of responses from a policy model (often an initial SFT model), followed by (2) the scoring of these responses by a separate reward model. This preference data is then (3) collected, filtered, and aggregated into a training batch. Finally, (4) a distributed fine-tuning step, commonly using an algorithm

Table 1. Comparison of off-the-shelf system solutions for LLM workflows.

System Class	Examples	Cross-Workflow Reuse	Heterogeneity/Topology Awareness	Elastic Scaling	DAG Abstraction
Workflow Engines	Argo, Airflow	No (Per-job isolation)	Limited (Node affinity)	Limited	Yes (Defines jobs)
Data Systems	Ray, Dask	No (Single-job scope)	Partial (Actor placement)	No (Static cluster)	Yes (Dynamic tasks)
LLM Serving Systems	vLLM, SGLang	No (Per-request opt.)	No (Single-node focus)	Partial (Model-level)	No (Stateless endpoint)
Container Orchestrator	Kubernetes	No (Opaque pods)	Limited (No topology)	Yes (Pod-level)	No (Pod is primitive)
FlowMesh (Our Work)	-	Yes	Yes	Yes (Service-level)	Yes (Workflow graph)

like Proximal Policy Optimization (PPO) (Schulman et al., 2017), updates the policy model’s weights. In a multi-tenant environment where multiple teams are iterating on variants of the same base model (e.g., Llama-3-8B), the SFT and reward model inference steps (1 and 2) are frequently identical across numerous concurrent experiments. These compute-intensive stages represent a prime target for deduplication.

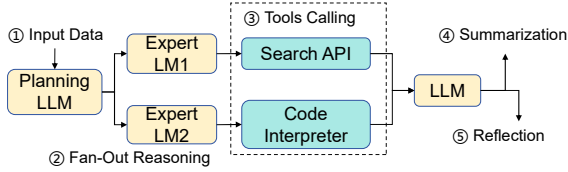


Figure 3. An agentic workflow orchestrates reasoning, tool use, and reflection.

Agentic Workflows: More complex agentic systems, such as those based on the ReAct paradigm (Yao et al., 2023), chain multiple model invocations, tool use, and reasoning steps (shown in Figure 3). For instance, an agent might involve a DAG where a planning LLM’s output is parsed, used to call an external tool (e.g., a search API or a code interpreter), and the tool’s output is then fed back into the LLM for summarization or reflection. In this architecture, the tool-use and summarization modules can be implemented as shared, stateless services shared by different agentic workflows, each pursuing distinct goals.

Prior Solutions and Limitations. To deploy these workloads and provide low-latency, low-cost service, prior solutions fail to address the specific, composite challenges of these new workloads. We categorize the shortcomings of four classes of existing systems in Table 1. Our analysis reveals that no existing solution holistically addresses the challenges. Workflow orchestrators define DAGs but lack multi-tenancy. Distributed frameworks like Ray are powerful for single, large jobs but operate within a static cluster allocation and do not share work across jobs. Specialized inference servers like vLLM and SGLang achieve high throughput for single models but are stateless endpoints unaware of the broader workflow context (Kwon et al., 2023b; Zheng et al., 2024). Finally, the default Kubernetes orchestrator lacks the semantic understanding of LLM workflows to perform intelligent placement, especially

concerning hardware heterogeneity and network topology.

Motivation and Ideas. Our analysis above distills into three foundational design principles for FlowMesh.

DAG abstraction with containerized execution. Each tenant’s workflow is parsed into a DAG, with each operator representing an LLM or algorithmic stage. Each operator is given a deterministic identity derived from its code, model/version, hyperparameters, and input lineage (Shen et al., 2025). This enables common-subgraph reuse: semantically identical sub-tasks (e.g., SFT on the same base checkpoint, reward-model inference over overlapping batches) are executed once and their materialized results are shared based on lineage, eliminating redundant computation and exposing reuse opportunities that job-scoped systems cannot see. To offset heterogeneity, we use a high-performance, containerized worker runtime; each local runtime wraps a state-of-the-art LLM runtime, such the vLLM engine that performs continuous batching.

Resource- and topology-aware job scheduling is performed at the operator granularity using a multi-objective cost model that accounts for compute demand, memory footprint, communication patterns, and SLOs. FlowMesh co-locates communication-heavy edges on high-bandwidth interconnects and binds model training to appropriate GPU classes. Crucially, it exploits *semantic* similarity to batch across tenants: compatible LLM invocations (policy rollouts, reward scoring, evaluation passes) are continuously batched through the worker runtime, improving throughput and tail latency. Topology awareness (intra-node vs. cross-WAN) and heterogeneity handling (L40/A100 mix) are explicit in the placement logic.

Serverless compute for elasticity and resilience. FlowMesh decouples the logical DAGs from physical resource capacity and manages worker pools elastically, scaling based on queue depths, cost targets, and SLOs. Tasks are lineage-backed, enabling deduplication, speculative re-execution, and fast recovery on preemption or failure; this borrows ideas from serverless and cloud-native computing to leverage stateless workers. Overall, FlowMesh’s serverless control plane converts static, ill-provisioned deployments into an adaptive fabric that matches real-time demand to distributed workers underneath.

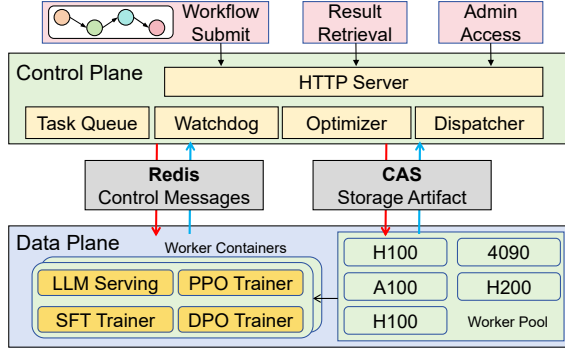


Figure 4. System overview of FlowMesh.

3 SYSTEM DESIGN

Overview. As shown in Figure 4, FlowMesh is architected as a dual-plane service fabric designed to improve the efficiency of multi-tenant, graph-structured LLM workflows with distributed workers. FlowMesh uses a DAG representation for the submitted workflow. Each operator in the workflow is assigned a unique, deterministic identity based on its model, parameters, and data lineage, enabling the system to automatically find and merge identical operators across different user workflows. These DAGs are managed by a centralized Control Plane, which employs a multi-objective, topology-aware scheduler that places operators on heterogeneous hardware and opportunistically batches compatible requests from different tenants to maximize GPU utilization. The execution itself is handled by a distributed Data Plane of stateless, containerized workers that are scaled elastically based on real-time demand. The combination of stateless workers and a lineage-backed DAG also provides inherent fault tolerance, allowing for simple, rapid recovery from failures. In the following sections, we elaborate on the details of each module.

DAG Abstraction. Prior work often models agentic workflows as DAGs, with each operator depicting LLM-based agent interaction, generic data processing or tool usage. Beyond this, we generalize this strategy to a broad class of LLM workflows, including operators for evaluation, data preparation, and RL-style post-training (e.g., SFT/DPO/PPO) (Rafailov et al., 2023). The system also maintains a queue of per-request DAGs for first-come-first-serve processing of the LLM workflows; each of them is compiled into its own DAG. Execution therefore proceeds by repeatedly taking ready operators from the DAGs in the queue and scheduling them when possible. By consolidating similar operators at dispatch time, the system achieves high utilization and low latency without compromising workflow isolation or provenance fidelity. This abstraction gives a clear lineage, reproducibility, and deduplication.

Cross-DAG consolidation. FlowMesh’s scheduler looks across the frontiers of many queued DAGs and performs the

following actions:

Unification by identity (deduplication). Each DAG is decomposed into fine-grained *operators*, which are the unit of reasoning and reuse. Every operator then receives a deterministic identity over its full execution context $H_{\text{task}} = \text{hash}(H_{\text{model}}, \text{canonical}(P), H_{\text{in},1}, \dots, H_{\text{in},n})$, where H_{model} digests the executor model, $\text{canonical}(P)$ is a serialization of hyperparameters and resource hints, and $H_{\text{in},i}$ hash the upstream datasets and prompts. If multiple ready operators from different DAGs share the same H_{task} , the system executes the computation at most once (or skips it entirely if the output already exists) and fans out the resulting artifact to each consumer DAG. This eliminates redundant compute while preserving each workflow’s structure.

Consolidation by execution signature (batching). When operators are not byte-identical but are compatible for joint execution (e.g., same executor and parameters, different inputs that can be microbatched), they are scheduled together as one batched run. Compatibility is captured by an execution signature $H_{\text{exec}} = \text{hash}(H_{\text{model}}, \text{canonical}(P), \text{resource_class})$, which deliberately omits the input hashes. The batched run produces per-input results that are routed back to the corresponding edges in their original DAGs. This improves throughput without conflating provenance. In both cases, each DAG remains a first-class, isolated object; we only unify executions, not the workflow graphs themselves.

Provenance and Isolation. All consolidations are gated by the above functions: *exact-match* (H_{task}) enables reuse while *compatible-match* (H_{exec}) enables batching inference tasks with the same LLM. Lineage is recorded per DAG edge, so prompts and intermediate outputs are referenced across workflows. Multi-tenant isolation also follows naturally from labeling the prompts and intermediate outputs and resolving the DAGs.

3.1 Control Plane: Routing and Online Batching

The control plane is the global coordinator of FlowMesh to decide where each operator should execute. Specifically, the control plane is responsible for: (i) placement on heterogeneous accelerators, (ii) cross-tenant batching of compatible work, and (iii) continuous admission of new requests into already-running executors. A key goal here is that these decisions are made under a single, unified objective, instead of via separate stages. At any instant, FlowMesh groups ready operators that invoke the same executor/model under the same hyperparameters and resource class. We denote this by H_{exec} as mentioned earlier, which omits the specific input payloads. All operators that share a given H_{exec} are batch-compatible: they can run on the same long-lived executor with the same loaded weights. We denote the current compatible set by $S(H_{\text{exec}})$.

Scheduling Objective. For each worker j and each candidate admitted batch $B \subseteq S(H_{\text{exec}})$ that j could serve, the control plane estimates a scalar utility

$$U(j, B) = w_t T_{\text{eff}}(j, B) - w_c C(j) + w_\ell G_{\text{loc}}(j, B). \quad (1)$$

Each term reflects one dimension of system behavior:

- $T_{\text{eff}}(j, B)$ is the predicted effective throughput of serving B on worker j (e.g., tokens/sec or samples/sec), given the model architecture, sequence lengths in B , and the GPU class of j . This pushes toward high accelerator utilization.
- $C(j)$ is the normalized cost rate of keeping worker j active (e.g., on-demand H100 vs. consumer GPU). This favors cheaper capacity when latency allows.
- $G_{\text{loc}}(j, B)$ is the locality gain from routing B to j : it is higher if j already has the needed model weights in GPU memory, the relevant adapters and tokenizers on local disk, or hot KV/cache state for that H_{exec} . This rewards reuse and minimizes network traffic.

The weights (w_t, w_c, w_ℓ) encode cluster policy. Latency-sensitive interactive evaluation can downweight $C(j)$, preferring “fast-but-expensive” GPUs. Large offline rollout generation can do the opposite and favor cheaper hardware, even at lower tokens/sec, as long as deadlines are met. Note that a pair (j, B) is *feasible* only if it satisfies hard constraints: (1) VRAM / memory bandwidth limits on j , (2) architectural requirements (e.g., instruction set, GPU generation), and (3) tenant affinity/anti-affinity (e.g., “must run on private GPUs”). Among feasible candidates, the control plane prefers the (j, B) that maximizes $U(j, B)$ in Eq. 1. Lastly, since the worker’s admission queue $Q_j(H_{\text{exec}})$ is shared, requests from different tenants that invoke the same model and parameters naturally co-reside on the same worker. This enables cross-DAG consolidation mentioned earlier in this section.

Online Admission and Continuous Batching. FlowMesh does not treat B as a one-shot job batch. Instead, each worker j maintains a live admission queue $Q_j(H_{\text{exec}})$ for every execution signature it is currently serving. The control plane continuously feeds new compatible operators from $S(H_{\text{exec}})$ into $Q_j(H_{\text{exec}})$, subject to feasibility and utility. In other words, B is the next slice of work to admit into an already running executor on j . This turns scheduling into continuous serving: multiple tenants’ requests stream into persistent executors that already have the correct model weights resident, and the runtime on j performs dynamic (token-by-token / step-by-step) batching across those requests. GPU occupancy stays high, and latency stays low, because the control plane can admit work immediately into

a hot lane rather than waiting to form a large static batch or spin up a fresh worker.

When the control plane admits work to $Q_j(H_{\text{exec}})$, it sends the operator spec(s), input content hashes, and the expected output schema. The worker’s continuous-batching runtime executes these requests, produces one output per logical input, and writes each output to the shared content-addressable store (CAS) under a fresh content hash. The control plane records those hashes and unblocks the downstream edges in each originating workflow DAG. Because every intermediate result is content-addressed, provenance is explicit.

Remark. Adding up the above designs, the control plane turns all tenant workflows into a single global stream of ready operators, continuously routes compatible requests into persistent executors, and uses one utility function (Eq. 1) to trade off throughput, cost, and data locality. It is simultaneously a scheduler, a batcher, and a multi-tenant serving router.

3.2 Data Plane: Elastic, Stateless Execution

The data plane is an elastic pool of stateless worker containers plus a shared content-addressable store (CAS) (Benet, 2014). Each worker behaves like a persistent serving executor for one or more execution signatures H_{exec} : once it has loaded the relevant model weights (and, if applicable, adapters, tokenizers, KV caches, etc.), the control plane keeps streaming compatible requests to it. Workers expose a continuous-batching runtime that can interleave requests from different tenants in real time.

Auto-scaling. Because workers are logically stateless, FlowMesh can scale them directly with demand: (1) If global queue depth or SLO pressure rises for some H_{exec} , new workers that can serve that signature are provisioned (e.g., new Pods in Kubernetes or newly leased GPU instances). (2) If demand falls and workers go idle, surplus workers are retired to avoid paying for idle accelerators. Joining the pool means starting to act as a serving lane for H_{exec} : the worker loads the needed model weights, initializes its continuous-batching runtime, and begins accepting admissions from the control plane. Leaving the pool means draining outstanding requests, flushing outputs to CAS, and exiting. If a worker fails mid-flight (e.g., preemption), any in-progress operators that did not complete are simply marked ready again and can be re-admitted elsewhere; completed outputs remain in CAS to simplify preemption.

CAS and Data Locality. All data artifacts, i.e., model checkpoints, adapters, tokenizers, rollout samples, reward scores, evaluation traces, summaries, live in an S3-compatible content-addressable store. Each artifact is named by a hash of its bytes. A worker serving H_{exec} operates as follows: (1) The control plane hands it new

requests by reference, i.e., via content hashes of required inputs. (2) The worker then resolves any missing inputs: it first checks its local cache (for weights, adapters, and recent inputs), then lazily fetches uncached artifacts from CAS. (3) The worker’s continuous-batching runtime executes those requests, potentially interleaving many tenants’ operators in a single forward/backward pass. (4) For each logical input, the worker emits an output artifact and writes it back to CAS under a new content hash.

Our solution has the following goals. First, locality emerges naturally: once a worker is “hot” for some H_{exec} , it tends to keep the right weights, KV cache state, and tokenizers resident, so future compatible requests are both faster (no reload) and cheaper (no repeated multi-GB transfers). The control plane captures this via $G_{\text{loc}}(j, B)$ in Eq. 1 and preferentially routes more of that H_{exec} traffic to that worker. Second, provenance is automatic: every intermediate is content-addressed, so downstream workflow stages receive immutable hashes rather than opaque in-memory pointers. This preserves isolation between tenants while still allowing cross-tenant reuse of identical results.

Remark. To conclude, the data plane is an autoscaled pool of persistent executors, each one a continuous-batching serving lane for one or more H_{exec} , backed by a shared CAS. Workers can appear, warm up, absorb multi-tenant traffic under tight SLOs, and disappear, without breaking lineage or forcing tenants to rewrite their workflows.

3.3 System Resilience

Resilience in FlowMesh emerges from three design choices: workers are stateless, and data is immutable and content-addressed. Instead of bespoke checkpointing, the system relies on idempotent re-execution with provenance that guarantees correctness. This follows the design principal of cloud-native computing. Each workflow remains its own DAG, while the control plane tracks per-DAG operator state (READY, RUNNING, COMPLETED) and exposes a pooled ready-operator queue across DAGs for scheduling and consolidation. When a worker fails or is preempted, any RUNNING operators it held are atomically returned to READY. A healthy worker then retrieves the same immutable inputs from the CAS and recomputes the result, making retry the common recovery path. Before re-executing a retried operator, the control plane checks whether the operator’s output already exists in the CAS, possibly produced by a consolidated run that served multiple DAGs. If it does, execution is skipped and the cached artifact is fanned back to the operator’s original edges, preserving provenance while eliminating duplicate work. Because workers carry no durable state and all inputs/outputs are CAS-backed, FlowMesh comfortably leverages preemptible or transient capacity. Autoscaling policies absorb bursts and

drain lull periods without correctness risk, turning inexpensive but unreliable instances into an operational advantage.

To mitigate stragglers and long-tail variance, the scheduler may launch speculative replicas. Only the first successful completion is published to the CAS; late or duplicate completions are discarded by content identity. This delivers at-most-once publication even under at-least-once execution. All fan-in and fan-out are expressed via content hashes, and each per-DAG edge records the exact versions consumed and produced. The result is precise prospective and retrospective provenance, enabling audit and exact replay.

4 IMPLEMENTATION

To demonstrate the flexibility of its core abstractions, FlowMesh is implemented to run on two fundamentally different infrastructure paradigms: a centrally managed Kubernetes cluster, typical of enterprise environments, and a decentralized, peer-to-peer compute marketplace, represented by Vast.ai. Table 2 summarizes the mapping of FlowMesh components to the primitives of each environment.

Deployment on Kubernetes Clusters. In a Kubernetes environment, FlowMesh is implemented as a set of custom controllers and a scheduler plugin, integrating natively with the cluster’s control plane (Portworx, 2024). The Planner, Router, and scheduler logic are packaged into a single Go-based binary that runs as a Kubernetes Deployment. It watches for FlowMeshWorkflow Custom Resource Definitions (CRDs) submitted by users. The scheduler is implemented as a Kubernetes scheduler plugin. When the controller identifies a ready task, it creates a placeholder Pod. The FlowMesh plugin intercepts this Pod and executes its Filter/Score logic to determine the optimal placement. The worker runtime is packaged as a Docker container and managed by a Kubernetes Deployment. A HorizontalPodAutoscaler is configured to scale the number of worker Pods based on the length of the global ready-task queue, exposed as a custom metric via Prometheus. The CAS is backed by a PersistentVolumeClaim that can be satisfied by any cluster-wide storage solution, such as CephFS, or a cloud provider’s object store.

Deployment on the Vast.ai Network. Deploying on Vast.ai requires adapting FlowMesh to a dynamic, heterogeneous, and geographically distributed P2P marketplace (AI, 2025). The control plane runs on a single, stable, on-demand Vast.ai instance. It uses the official Vast.ai Python CLI and SDK to interact with the marketplace programmatically (Vast.ai, 2024a;b). The Kubernetes HPA is replaced by a custom control loop. This loop periodically queries the ready-task queue. To scale up, it formulates a query based on a task’s requirements and executes `vastai search` offers to

Table 2. Mapping of FlowMesh Components to Infrastructure Primitives

Architectural Component	Implementation on A Centralized K8s Cluster	Implementation on Decentralized Vast.ai
Control Plane	Kubernetes Deployment with custom controller	Runs on a stable, on-demand Vast.ai instance
Worker Provisioning	HorizontalPodAutoscaler (HPA)	Custom controller making API calls
Task Dispatch	Controller creates a Kubernetes Pod	Control plane sends task spec via SSH/startup script
Storage (CAS)	PersistentVolume backed by cluster storage	External S3-compatible object store
Heterogeneity Info	Kubernetes node labels	Real-time query results from Vast.ai
Cost Model Input	Pre-configured node costs	Dynamic, real-time prices from Vast.ai

find suitable instances. The scheduler’s cost model directly ingests the dynamic pricing and host reliability metrics returned by the API. It then rents the best-scoring machine using `vastai create instance`. To scale down, the loop identifies idle workers and terminates their instances using `vastai destroy instance`. Since Vast.ai instances have ephemeral storage, the CAS must be an external, S3-compatible object store. When dispatching a task to a newly created worker, the control plane passes the task specification, including input artifact hashes and temporary credentials for the object store, via environment variables or a startup script configured in the instance creation command (Vast.ai, 2024c).

Containerized Workers. Each worker is a Docker image that wraps a state-of-the-art engine (e.g., vLLM) behind a thin agent: it pulls inputs by content hash from the CAS, executes, and pushes immutable outputs, keeping workers stateless and interchangeable. On Kubernetes, the same image runs as a deployment and scales with queue depth; placement filters on GPU type/VRAM. On Vast.ai, the control loop leases instances, pulls the identical image, and starts the container with read-only CAS credentials. A single CUDA 12.8 image supports heterogeneous GPUs: GPU-aware runtimes (PyTorch) select kernels by compute capability; devices are injected via `nvidia-container-runtime`, covering A100/H100 and prosumer RTX GPUs. Images are published to a private registry with regional mirrors; Kubernetes pulls via `ImagePullSecrets` and Vast.ai via short-lived tokens, while the S3-compatible CAS remains the source of truth for weights, adapters, tokenizers, and intermediates.

5 EVALUATION

We conduct an empirical evaluation to justify FlowMesh’s design and evaluate its performance. Our experiments are guided by the following questions: (1) How does FlowMesh perform in serving concurrent LLM workflows compared to prior solutions? (2) How does each component contribute to FlowMesh, and how well does it behave in various deployment scenarios, including failures? (3) How does FlowMesh scale and integrate with different runtime backends?

In this section, we first analyze throughput using primitive workflows, followed by cost and energy usage comparisons. We then present ablation and robustness analyses, and conclude with scalability experiments.

5.1 Experimental Setup

LLM Workflows. We use four primitive LLM post-training workflows spanning common deployment tasks.

- *Agentic (Inference) Workflow:* multiple agentic workflows covering various model and dataset configurations.
- *Supervised Fine-Tuning (SFT) Workflow* for both LoRA SFT and full-weights SFT, followed by an evaluation pipeline and a hold-out test set.
- *Direct Preference Optimization (DPO) Workflow* for RL training using configured preference pairs, followed by an evaluation job.
- *Proximal Policy Optimization (PPO) Workflow* for RL training, followed by an evaluation job.

We use public datasets including GSM8K (Cobbe et al., 2021), MMLU (Hendrycks et al., 2021), and TruthfulQA (Lin et al., 2022) in these workflows.

Test Workloads are divided into two groups:

- *Group A* consists of five agentic inference workflows with diverse DAG topologies, including reasoning chains, retrieval-augmented generation, and multi-agent collaboration tasks. These workflows are designed to evaluate FlowMesh’s ability to serve heterogeneous inference pipelines efficiently. We sample 200 workflows from these datasets to construct the test workload.
- *Group B* extends Group A by adding four post-training pipelines following RLHF/RLAIF methodologies. Each pipeline comprises typical sub-stages such as SFT, PPO, and DPO, enabling us to assess scheduling, dependency management, and resource reuse under multi-stage fine-tuning workloads. We sample 200 workflows for training and 100 workflows for evaluating inference from GSM8K, respectively, to construct the test workload.

Baselines. We compare FlowMesh against several baselines; they are categorized in whether to break down the workflow (decompose or monolithic) and how to schedule the jobs

(first-fit, static routing, or consolidate):

- *MF: Monolithic + First-Fit:* We use vLLM + TRL (Kwon et al., 2023a; von Werra et al., 2023) and implement a simple first-fit heuristic to place the next entire workflow on the first available worker.
- *DS: Decompose + Static:* We implement JellyBean (Wu et al., 2022) that similarly decomposes complex ML workflows into operators and allocates them onto distributed workers using a static routing based on the operator’s functionality; e.g., inference and SFT workers always do their designated jobs.
- *DR: Decompose + Round-Robin:* similar to DS, we decompose ML workflows but allocate the operators using a round-robin assignment to all available workers.
- *FlowMesh: Decompose + Consolidate:* Our solution decomposes the workflows, consolidates jobs by the operators, and allocates them to distributed workers.

The workflows utilize base models including Llama-3.1-8B, Llama-3.2-3B, and Llama-3.2-1B (Meta AI, 2024). The experiments are conducted on a local cluster (Section 5.2, 5.3), Kubernetes and Vast.ai (Section 5.4). All the baselines and workers are implemented with vLLM for inference and Huggingface TRL for RL training (Kwon et al., 2023a; von Werra et al., 2023). The batch size is set to 24 for Group A and 12 for Group B, respectively.

5.2 Performance and Cost Analyses

Cloud-based serving platforms inherently balance cost efficiency, monetary cost, and serving performance. We first evaluate FlowMesh from the perspective of cost and energy efficiency, and then analyze its end-to-end serving latency and throughput scalability.

Experiment Metrics and Setup. Following (Roloff et al., 2017), we assess both monetary cost and energy consumption under realistic workloads, and use two composite metrics, the Cost–Delay Product (CDP) and the Energy–Delay Product (EDP), to jointly measure the trade-offs between efficiency and latency: $CDP = \frac{\text{Total Cost}}{\text{\#Tasks}} \times \text{AvgTime}$, $EDP = \frac{\text{Total Energy}}{\text{\#Tasks}} \times \text{AvgTime}$, where lower values indicate more cost- and energy-efficient results. We also report average task latency that aggregates all sub-stages within a workflow DAG (e.g., PPO fine-tuning, reasoning chains).

The experiment is conducted on 6 GPU workers, including Nvidia H100 NVL (94 GB), RTX 4090 (48 GB), and RTX 4090 (24 GB) in equal proportion. Unit prices (\$/hour) reflect average rental rates from Vast.ai in Oct 2025. Each worker samples instantaneous power through periodic heartbeats, while the host integrates power over time to compute per-task energy usage. To simulate realistic peak-and-tail traffic, we apply an exponentially decaying query rate from

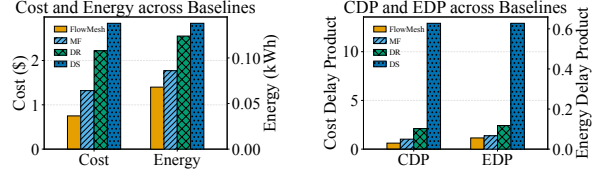


Figure 5. FlowMesh compared with baselines. Left: Total cost and energy consumption under identical workload conditions. Right: Cost–Delay Product (CDP) and Energy–Delay Product (EDP).

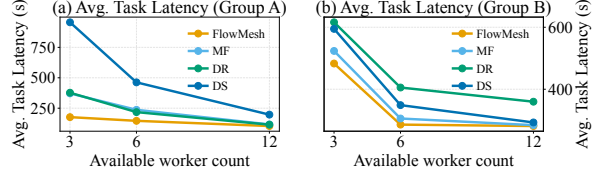


Figure 6. Average task latency with different numbers of available workers. FlowMesh achieves similar or better latency in all cases.

6 to 0.6 qpm, with request workflows drawn randomly from Group A workloads.

Comparisons. Figure 5 shows that FlowMesh delivers the lowest cost and energy use among all baselines. It reduces monetary cost by 1.8×–3.8× and energy usage by 1.3×–2.0×, while achieving 2×–10× better CDP and EDP. Although DS attains slightly higher throughput by favoring H100 GPUs, FlowMesh instead makes better use of smaller GPUs, trading some throughput for substantially improved cost and energy efficiency.

Results in Figure 6 indicate that FlowMesh consistently matches or outperforms all baseline schedulers in latency. The advantage is more significant with fewer workers, where baselines often queue for resources, while FlowMesh consolidates common tasks across workflows, thus skipping the queue. As more workers become available, everyone’s latency improves, the queueing pressure eases, and the gap narrows; this is as expected. Even then, FlowMesh maintains comparable or slightly better latency.

Overall, our experiments verify that FlowMesh achieves better cost and energy efficiency compared to baseline solutions; meanwhile, it keeps the latency comparable or even better relative to running the workflows as a whole piece. These gains stem from elastic resource allocation and improved scheduling.

5.3 Ablation and Robustness Study

Ablation Study. As shown in Table 3, we ablate three components in FlowMesh to quantify their contributions, thus justifying the design of our solution.

Table 3. Ablation study results showing latency and cost improvements of each component.

Disable Component	Latency ($\times$)	Cost ($\times$)
Operator Consolidation	1.36 $\times$	1.25 $\times$
Elastic Resource Usage	1.21 $\times$	1.78 $\times$
Multi-Objective Scheduling	1.33 $\times$	1.24 $\times$
FlowMesh (full)	1.00 $\times$	1.00 $\times$

Table 4. Robustness test under two representative scenarios.

Error	Avg. Latency $\uparrow$	Detection Time
Worker Crash	13.3%	30.0s
Wrong Resource Spec	5.1%	8.6s

Operator Consolidation. Batching and executing homogeneous sub-tasks amortize repeated setup and warm-up (e.g., model/context loading). On a batch of 24 concurrent agent workflows with the device mix in §5.2, disabling task consolidation increases average latency by 1.36 $\times$ and total running cost by 1.25 $\times$. Benefits grow with higher concurrency as merge opportunities increase.

Elastic Resource Usage. Adapting the worker pool to the actual workload (pausing idle workers, reactivating on demand) cuts long-tail waste. Under the same setup, disabling elastic resource usage increases average latency by 1.21 $\times$ and total cost by 1.78 $\times$.

Multi-Objective Scheduling. A customizable objective (cost-first vs. performance-first) matches ready tasks to eligible workers. Compared with Round-Robin with random allocation, our full solution improves service efficiency by 1.33 $\times$ and reduces cost by 1.24 $\times$. We observe larger gains when the objective can be tuned specifically to each testing workload, although we did not report this here.

Robustness. We verify the robustness of FlowMesh on heterogeneous, distributed hardware with two representative failure scenarios using the §5.2 setup.

Worker Crash. To emulate unexpected outages (e.g., host, network, or power faults), we forcibly take one H100 worker offline after 2 minutes. FlowMesh uses a *watchdog* that couples worker health monitoring with dispatch. Each worker is expected to emit periodic heartbeats; when heartbeats are missed for an entire watchdog period (30s), the system marks the worker as failed, immediately re-enqueues any tasks that were in flight on that worker, and reschedules their dependent sub-tasks. The results show that FlowMesh continues the execution and completes all affected tasks with only a moderate increase in end-to-end latency (Table 4).

Wrong Resource Specification. Users specify resource demand (e.g., GPU memory requirement) using hint annota-

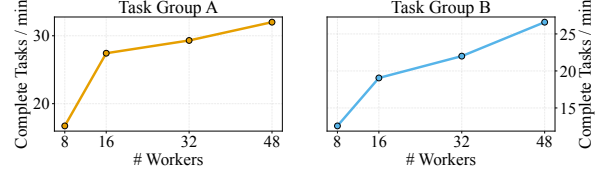


Figure 7. Complete tasks under different numbers of workers on a Kubernetes cluster.

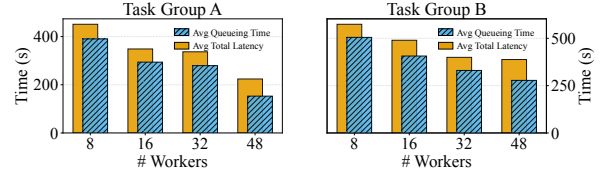


Figure 8. Average queueing time and total latency under different numbers of workers on a Kubernetes cluster.

tions to guide the placement. We intentionally under-specify the GPU memory requirement in a multi-stage inference workflow. Similarly, the worker proactively reports the failure due to a resource shortage. The control plane resubmits the faulty task to the next available worker with sufficient resources. As a result, the workflow still completes successfully, and only a small latency loss is observed.

5.4 Scalability Study

Scalability on Kubernetes. To evaluate FlowMesh’s scalability on large-scale clusters, we use the experiment in §5.2 on a centralized Kubernetes cluster. The experimental cluster consists of up to 48 GPU nodes, each with one NVIDIA H100 (80 GB HBM3) GPU. The cluster is orchestrated using `kubect1 v1.32.1` (Kustomize v5.5.0). The workload remains the same, while the number of concurrent submissions is scaled to 64 to simulate a high-concurrency serving scenario with multiple tenants.

Figure 7 shows that increasing the number of workers improves the throughput sub-linearly. Specifically, in Group A, FlowMesh completes 17 tasks per minute from the queue with 8 workers, while completes 32 tasks per minute with 48 workers; in Group B, 13 tasks with 8 workers and 26 tasks per minute with 48 workers. On the other hand, Figure 8 shows that increasing the number of workers reduces overall latency as the cluster scales. Specifically, FlowMesh reduces the queueing time, e.g., from 400 sec with 8 workers to 150 sec with 48 workers for Group A workload, while offering a similar per-workflow latency profile.

In both scenarios, we observe a similar situation as in Figure 6; queueing delay accounted for a large portion of the entire processing time. Indeed, the overhead caused by

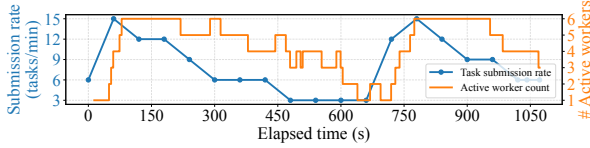


Figure 9. Dynamic workload (new tasks/min) and the number of active workers in FlowMesh over time.

Kubernetes is still sizable, but improving Kubernetes and adapting FlowMesh to it falls outside the scope of this paper; we will defer this to future work. Nevertheless, these results indicate that FlowMesh can scale on a larger cluster.

Elasticity on Vast.ai. To validate elasticity of FlowMesh, we conduct a dynamic-load experiment where the task arrival rate varies over time and track the number of active workers, using the experiment in §5.2. Figure 9 show the results, where the system effectively scales up and down when incoming workflows vary. A mild lag of 30-60 seconds is observed in elastic node creation and closure; it is attributed to the Vast.ai instance bidding API. Nonetheless, we consider this delay acceptable in practice.

6 RELATED WORK

LLM Serving Systems A number of recent systems focus on optimizing the *inference* serving of large language models. vLLM (Kwon et al., 2023b) introduces a PagedAttention mechanism for dynamic memory management of the key-value cache, enabling efficient batching of autoregressive decoding. By reducing memory fragmentation and allowing cache reuse, vLLM achieves 2–4× higher throughput at similar latency compared to prior solutions like NVIDIA’s FasterTransformer (NVIDIA Corporation, 2022) and Orca (Yu et al., 2022). Orca is a distributed serving system that pioneered iteration-level scheduling and selective batching to better utilize GPU capacity during multi-token generation. Other frameworks such as DeepSpeed-Inference (Aminabadi et al., 2022) apply optimized kernels and parallelism strategies to maximize throughput on hardware accelerators. These systems substantially improve runtime efficiency for deploying a trained LLM. However, their focus is on the *inference phase* of a single model. In contrast, our work targets the *post-training workflow* that produces and evaluates new model versions.

Workflow Orchestration Our work is related to general-purpose distributed execution frameworks that manage machine learning workflows. Ray (Moritz et al., 2018) supports a dynamic task graph and an actor-based abstraction to handle heterogeneous workloads. Dask (Rocklin, 2015) provides a dynamic scheduler for parallelizing Python computations, and Modin (Petersohn et al., 2020) scales data preprocessing via distributed DataFrame operations. Alpa

(Zheng et al., 2022) automates distributed parallelization of deep learning models. JellyBeanDS (Wu et al., 2022) deploys complex ML workflows on a hybrid cloud with heterogeneous workers, optimizing for combined compute and networking costs. These frameworks are general and require users to manually stitch components of a complex LLM post-training pipeline. In contrast, our system provides an integrated infrastructure tailored for LLM post-training workflows.

LLM Post-Training Workflows After an LLM’s initial pre-training, several techniques improve or adapt the model. Supervised fine-tuning (SFT) is used in FLAN and T0 (Wei et al., 2022; Sanh et al., 2022), as well as in the initial stages of InstructGPT (Ouyang et al., 2022). However, SFT may not capture nuanced human preferences, motivating RLHF (Christiano et al., 2017), which optimizes an LLM’s behavior using a reward model trained on human preferences. To reduce the human labeling burden, RLAIIF (Lee et al., 2024) and Constitutional AI (Bai et al., 2022) propose using AI-generated feedback. Another approach is agentic workflows, where LLMs act as decision-makers. ReAct (Yao et al., 2023) and HuggingGPT (Shen et al., 2023) represent such pipelines by combining reasoning and acting or orchestrating multiple models. While these works contribute to model improvement, our focus is on creating a shared infrastructure to support such workflows efficiently and at scale.

7 CONCLUSION

In this work, we presented FlowMesh, a system that treats post-training and LLM-agent workloads as a shared, elastic service rather than isolated jobs. By unifying redundant work, batching compatible tasks, and scheduling them across diverse accelerators, FlowMesh improves efficiency and reduces cost, while its global control plane and stateless data plane enable scalable, portable, and resilient operation under real-world workloads.

REFERENCES

- AI, S. In-Depth Analysis of Vast.ai: The Cloud Computing Platform AI Users Can’t Ignore. *Skywork AI Blog*, June 2025.
- Aminabadi, R. Y., Rajbhandari, S., Zhang, M., Awan, A. A., Li, C., Li, D., Zheng, E., Ruwase, O., Smith, S., Rasley, J., and He, Y. Deepspeed-inference: Enabling efficient inference of transformer models at unprecedented scale. In *International Conference for High Performance Computing, Networking, Storage and Analysis (SC 2022)*, pp. 46:1–46:15. IEEE, 2022.
- Bai, Y., Kadavath, S., Kundu, S., Askell, A., Ganguli, D.,

- et al. Constitutional AI: Harmlessness from AI feedback. *arXiv preprint arXiv:2212.08073*, 2022.
- Benet, J. Ipfs - content addressed, versioned, p2p file system, 2014. URL <https://arxiv.org/abs/1407.3561>.
- Christiano, P. F., Leike, J., Brown, T. B., Martic, M., Legg, S., and Amodei, D. Deep reinforcement learning from human preferences. In *Advances in Neural Information Processing Systems 30 (NeurIPS 2017)*, 2017.
- Cobbe, K., Kosaraju, V., Bavarian, M., Chen, M., Jun, H., Kaiser, L., Plappert, M., Tworek, J., Hilton, J., Nakano, R., Hesse, C., and Schulman, J. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.
- Gu, J., Jiang, X., Shi, Z., Tan, H., Zhai, X., Xu, C., Li, W., Shen, Y., Ma, S., Liu, H., et al. A survey on llm-as-a-judge. *arXiv preprint arXiv:2411.15594*, 2024.
- Hendrycks, D., Burns, C., Basart, S., Zou, A., Mazeika, M., Song, D., and Steinhardt, J. Measuring massive multitask language understanding. *Proceedings of the International Conference on Learning Representations (ICLR)*, 2021.
- Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., and Chen, W. Lora: Low-rank adaptation of large language models. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2022. URL <https://arxiv.org/abs/2106.09685>. Poster session.
- Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J., Zhang, H., and Stoica, I. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th ACM Symposium on Operating Systems Principles (SOSP 2023)*, pp. 611–626, 2023a.
- Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J. E., Zhang, H., and Stoica, I. Efficient memory management for large language model serving with pagedattention. In *29th ACM Symposium on Operating Systems Principles (SOSP 2023)*, 2023b.
- Lee, H., Phatale, S., Mansoor, H., Mesnard, T., Ferret, J., Lu, K., Bishop, C., Hall, E., Carbune, V., Rastogi, A., and Prakash, S. Rlaif vs. rlhf: Scaling reinforcement learning from human feedback with ai feedback. In *Proceedings of the 41st International Conference on Machine Learning (ICML 2024)*, volume 235 of *Proceedings of Machine Learning Research*, pp. 26874–26901. PMLR, 2024.
- Lin, S., Hilton, J., and Evans, O. Truthfulqa: Measuring how models mimic human falsehoods, 2022. URL <https://arxiv.org/abs/2109.07958>.
- Meta AI. The llama 3 herd of models. <https://arxiv.org/abs/2407.21783>, 2024. Accessed: 2025-10-27.
- Moritz, P., Nishihara, R., Wang, S., Tumanov, A., Liaw, R., Liang, E., Elibol, M., Yang, Z., Paul, W., Jordan, M. I., and Stoica, I. Ray: A distributed framework for emerging AI applications. In *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18)*, pp. 561–577, Carlsbad, CA, 2018. USENIX Association.
- NVIDIA Corporation. FasterTransformer: A library for accelerating transformer inference. <https://github.com/NVIDIA/FasterTransformer>, 2022.
- Ouyang, L., Wu, J., Jiang, X., Almeida, D., Wainwright, C. L., Mishkin, P., Zhang, C., Agarwal, S., Slama, K., Ray, A., Schulman, J., Hilton, J., Kelton, F., Miller, L., Simens, M., Askell, A., Welinder, P., Christiano, P. F., Leike, J., and Lowe, R. Training language models to follow instructions with human feedback. In *Advances in Neural Information Processing Systems 35 (NeurIPS 2022)*, 2022.
- Petersohn, D., Macke, S., Xin, D., Ma, W. W., Lee, D. J. L., Mo, X., Gonzalez, J. E., Hellerstein, J. M., Joseph, A. D., and Parameswaran, A. G. Towards scalable dataframe systems. *Proceedings of the VLDB Endowment*, 13(11): 2033–2046, 2020.
- Portworx. The Role of Kubernetes in AI: A Comprehensive Guide. *Portworx Knowledge Hub*, 2024.
- Rafailov, R., Sharma, A., Mitchell, E., Ermon, S., and Finn, C. Direct preference optimization: Your language model is secretly a reward model. *arXiv preprint arXiv:2305.18290*, 2023. URL <https://arxiv.org/abs/2305.18290>.
- Rocklin, M. Dask: Parallel computation with blocked algorithms and task scheduling. In *Proceedings of the 14th Python in Science Conference (SciPy 2015)*, pp. 126–132. SciPy, 2015.
- Roloff, E., Diener, M., Carreño, E. D., Moreira, F. B., Gaspar, L. P., and Navaux, P. O. Exploiting price and performance tradeoffs in heterogeneous clouds. In *Companion Proceedings of The 10th International Conference on Utility and Cloud Computing, UCC '17 Companion*, pp. 71–76, New York, NY, USA, 2017. Association for Computing Machinery. ISBN 9781450351959. doi: 10.1145/3147234.3148103. URL <https://doi.org/10.1145/3147234.3148103>.
- Sanh, V., Webson, A., Raffel, C., Bach, S. H., Sutawika, L., Alyafeai, Z., Chaffin, A., Stiegler, A., Scao, T. L., Raja, A., Dey, M., Bari, M. S., Xu, C., Thakker, U., Sharma,

- S., Szczechla, E., Kim, T., Chhablani, G., Nayak, N. V., Datta, D., Chang, J., Jiang, M. T., Wang, H., Manica, M., Shen, S., Yong, Z. X., Pandey, H., Bawden, R., Wang, T., Neeraj, T., Rozen, J., Sharma, A., Santilli, A., Févry, T., Fries, J. A., Teehan, R., Biderman, S., Gao, L., Bers, T., Wolf, T., and Rush, A. M. Multitask prompted training enables zero-shot task generalization. In *International Conference on Learning Representations (ICLR)*, 2022.
- Schulman, J., Wolski, F., Dhariwal, P., Radford, A., and Klimov, O. Proximal policy optimization algorithms. In *Proceedings of the 34th International Conference on Machine Learning (ICML)*. PMLR, 2017. URL <https://arxiv.org/abs/1707.06347>.
- Shen, J., Wadlom, N., and Lu, Y. Batch query processing and optimization for agentic workflows, 2025. URL <https://arxiv.org/abs/2509.02121>.
- Shen, Y., Song, K., Tan, X., Li, D., Lu, W., and Zhuang, Y. HuggingGPT: Solving AI tasks with ChatGPT and its friends in Hugging Face. In *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*, 2023.
- Vast.ai. Vast.ai CLI Documentation. <https://docs.vast.ai/cli/>, 2024a.
- Vast.ai. Vast.ai Python SDK. <https://github.com/vast-ai/vast-sdk>, 2024b.
- Vast.ai. Linux Virtual Machines on Vast.ai. <https://docs.vast.ai/linux-virtual-machines>, 2024c.
- Verma, A., Pedrosa, L., Korupolu, M., Oppenheimer, D., Tune, E., and Wilkes, J. Large-scale cluster management at google with borg. In *Proceedings of the European Conference on Computer Systems (EuroSys)*, pp. 1–17. ACM, 2015. doi: 10.1145/2741948.2741964. URL <https://research.google.com/pubs/archive/44843.pdf>.
- von Werra, L., Belkada, Y., Tunstall, L., Beeching, E., Thrush, T., Lambert, N., Huang, S., Rasul, K., and Gallouédec, Q. Trl: Transformer reinforcement learning. <https://github.com/huggingface/trl>, 2023.
- Wei, J., Bosma, M., Zhao, V. Y., Guu, K., Yu, A. W., Lester, B., Du, N., Dai, A. M., and Le, Q. V. Finetuned language models are zero-shot learners. In *International Conference on Learning Representations (ICLR)*, 2022.
- Wu, Y., Lentz, M., Zhuo, D., and Lu, Y. Serving and optimizing machine learning workflows on heterogeneous infrastructures. *arXiv preprint arXiv:2205.04713*, 2022.
- Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., and Cao, Y. React: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023.
- Yu, G., Jeong, J. S., Kim, G., Kim, S., and Chun, B. Orca: A distributed serving system for Transformer-Based generative models. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*, pp. 521–538, Carlsbad, CA, 2022. USENIX Association.
- Zaharia, M., Chowdhury, M., Das, T., Dave, A., Ma, J., McCauley, M., Franklin, M. J., Shenker, S., and Stoica, I. Resilient distributed datasets: A fault-tolerant abstraction for in-memory cluster computing. In *Proceedings of the 9th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, pp. 15–28. USENIX Association, 2012. URL <https://www.usenix.org/conference/nsdi12/technical-sessions/presentation/zaharia>.
- Zheng, L., Li, Z., Zhang, H., Zhuang, Y., Chen, Z., Huang, Y., Wang, Y., Xu, Y., Zhuo, D., Xing, E. P., Gonzalez, J. E., and Stoica, I. Alpa: Automating inter- and intra-operator parallelism for distributed deep learning. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*, pp. 559–576, Carlsbad, CA, 2022. USENIX Association.
- Zheng, L., Yin, L., Xie, Z., Sun, C., Huang, J., Yu, C. H., Cao, S., Kozyrakis, C., Stoica, I., Gonzalez, J. E., Barrett, C., and Sheng, Y. Sglang: Efficient execution of structured language model programs. In *Proceedings of the 38th Conference on Neural Information Processing Systems (NeurIPS 2024)*, 2024. URL <https://arxiv.org/abs/2312.07104>. Preprint version available on arXiv.